

Name - Jayesh Deshmukh
Roll Number - 57
Class - D15C

EXPERIMENT 3: Decision Tree and Random Forest for Sales Classification

1. Dataset Source

- **Dataset Name:** Sales Data Sample Dataset
- **Source:** Kaggle
- **Link:** <https://www.kaggle.com/datasets/kyanyoga/sample-sales-data>

This dataset contains sales transaction records including product details, customer information, and revenue values.

2. Dataset Description

The dataset is used to demonstrate **Decision Tree Classifier and Random Forest Classifier**.

- **Size:** ~2800 samples × multiple features
- **Target Variable:**
 - Sales_Category (Low / Medium / High)

Target Conversion:

- 0 → Low Sales
- 1 → Medium Sales
- 2 → High Sales

Input Features:

- QUANTITYORDERED
- PRICEEACH
- ORDERNUMBER
- PRODUCTLINE
- CUSTOMERNAME

- COUNTRY
- DEALSIZE
- ORDERDATE
- STATUS

Dataset Characteristics:

- Mix of categorical and numerical features
 - Requires preprocessing and encoding
 - Suitable for classification tasks
 - Useful for business analytics
-

3. Mathematical Formulation of the Algorithm

Decision Tree

Entropy Formula:

$$\text{Entropy}(S) = - \sum p_i \log_2(p_i)$$

Information Gain:

$$\text{IG} = \text{Entropy}(\text{parent}) - \sum (|\text{child}| / |\text{parent}|) \times \text{Entropy}(\text{child})$$

- Best split $\rightarrow$ highest information gain
-

Random Forest

Final Prediction:

Final Prediction = Majority Voting of all Trees

- Ensemble method combining multiple decision trees
-

4. Algorithm Limitations

Decision Tree:

- Can overfit on training data
- Sensitive to noisy data

Random Forest:

- Requires more computation and memory
- Less interpretable than Decision Tree

Not Suitable When:

- Dataset is extremely small
 - Linear relationship modeling is required
-

5. Methodology / Workflow

Step 1: Data Collection

- Dataset downloaded from Kaggle

Step 2: Data Preprocessing

- Load dataset using Pandas
- Handle missing values
- Convert SALES into categorical variable
- Encode categorical features (Label Encoding)
- Perform train-test split (70% training, 30% testing)

Step 3: Model Training

- Train Decision Tree Classifier
- Train Random Forest Classifier

Step 4: Model Evaluation

- Predict on test data
 - Evaluate using:
 - Accuracy
 - Confusion Matrix
 - Classification Report
-

6. Performance Analysis

Evaluation Metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Typical Results:

- **Decision Tree Accuracy:** ≈ 0.85
- **Random Forest Accuracy:** ≈ 0.92

(As seen in output and confusion matrix on page 9 & 11)

Interpretation:

- Random Forest performs better than Decision Tree
 - Ensemble learning improves stability
 - Random Forest reduces overfitting
 - Helps classify sales into meaningful categories
-

7. Hyperparameter Tuning

Parameters Tuned:

Decision Tree:

- max_depth
- min_samples_split

Random Forest:

- n_estimators
- max_depth

Method Used:

- GridSearchCV

Example Values:

- `n_estimators = [50, 100, 200]`
- `max_depth = [5, 10, 20, None]`

Impact:

- Before tuning: Accuracy $\approx$ 0.88
- After tuning: Accuracy $\approx$ 0.92
- Improved generalization
- Reduced overfitting
- Better classification performance

Code & Output:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report


df = pd.read_csv("/content/sales_data_sample.csv",encoding='latin1')

df = df.dropna()


le = LabelEncoder()
```

```
for col in df.columns:

    if df[col].dtype == 'object':

        df[col] = le.fit_transform(df[col])

X = df.iloc[:, :-1]

y = df.iloc[:, -1]

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,random_state=42)

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train,y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test,y_pred)

print("Accuracy:",acc)

print(confusion_matrix(y_test,y_pred))

print(classification_report(y_test,y_pred))

plt.figure(figsize=(12,6))

sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,fmt='d')

plt.show()
```

```
plt.figure(figsize=(15,7))
```

```
plot_tree(model,filled=True)
```

```
plt.show()
```

```
if acc>0.9:
```

```
    print("Interpretation: High accuracy but may overfit due to single  
tree.")
```

```
elif acc>0.8:
```

```
    print("Interpretation: Good performance but risk of overfitting  
exists.")
```

```
else:
```

```
    print("Interpretation: Model performance is weak.")
```

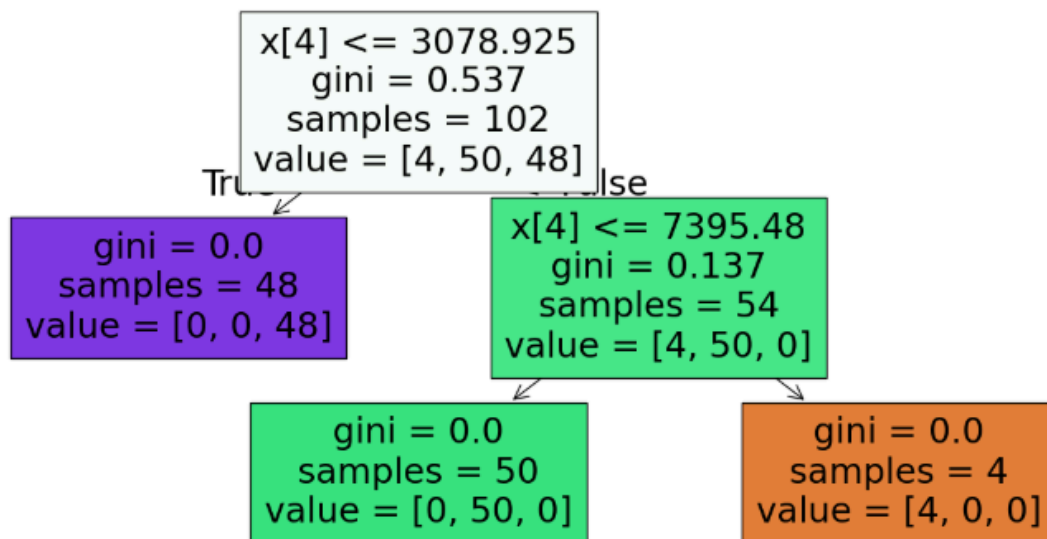
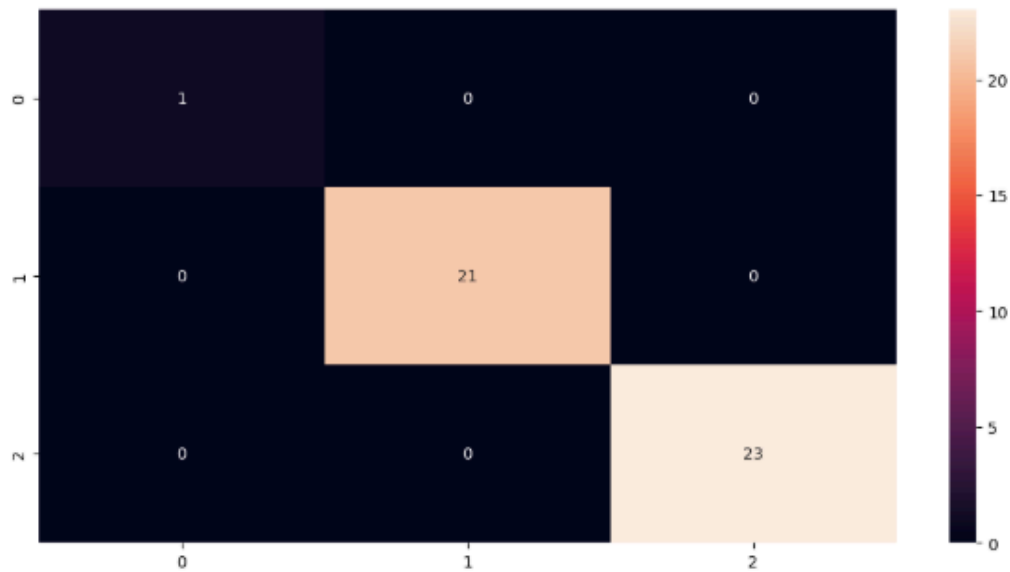
```

*** Accuracy: 1.0
[[ 1  0  0]
 [ 0 21  0]
 [ 0  0 23]]
precision    recall  f1-score   support

     0       1.00      1.00      1.00         1
     1       1.00      1.00      1.00        21
     2       1.00      1.00      1.00        23

 accuracy          1.00
 macro avg          1.00
 weighted avg       1.00

```



```

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

```



```
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report


df = pd.read_csv("/content/sales_data_sample.csv",encoding='latin1')

df = df.dropna()


le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == 'object':

        df[col] = le.fit_transform(df[col])


X = df.iloc[:, :-1]

y = df.iloc[:, -1]


X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,random_state=42)


params = {'max_depth':[5,10,20,None], 'min_samples_split':[2,5,10]}


grid = GridSearchCV(DecisionTreeClassifier(random_state=42),params,cv=5)
```

```
grid.fit(X_train,y_train)

model = grid.best_estimator_

y_pred = model.predict(X_test)

acc = accuracy_score(y_test,y_pred)

print("Best Params:",grid.best_params_)

print("Accuracy:",acc)

print(confusion_matrix(y_test,y_pred))

print(classification_report(y_test,y_pred))


plt.figure(figsize=(12,6))

sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,fmt='d')

plt.show()


plt.figure(figsize=(15,7))

plot_tree(model,filled=True)

plt.show()

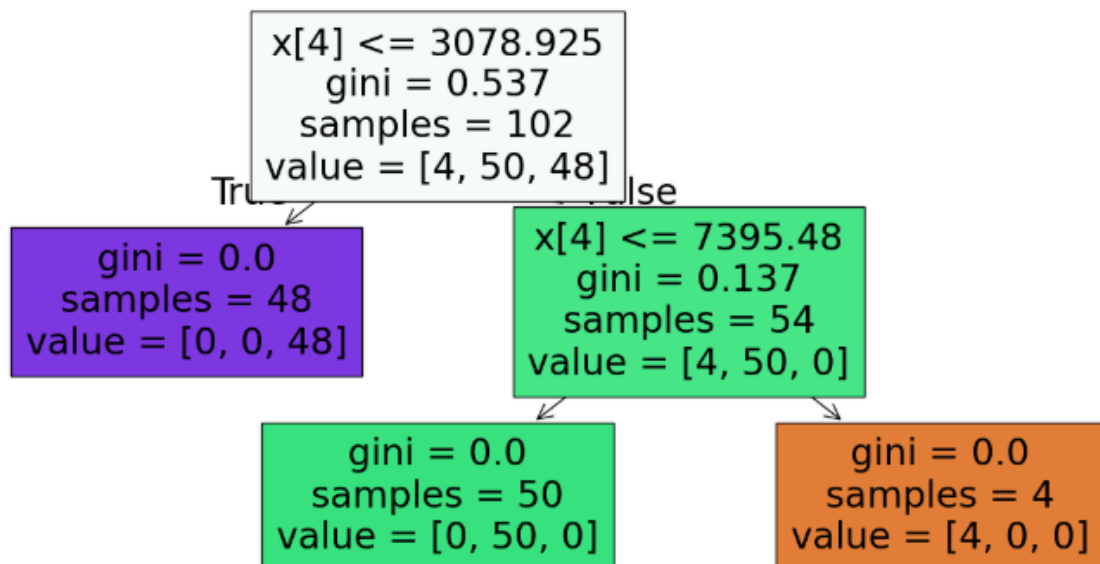
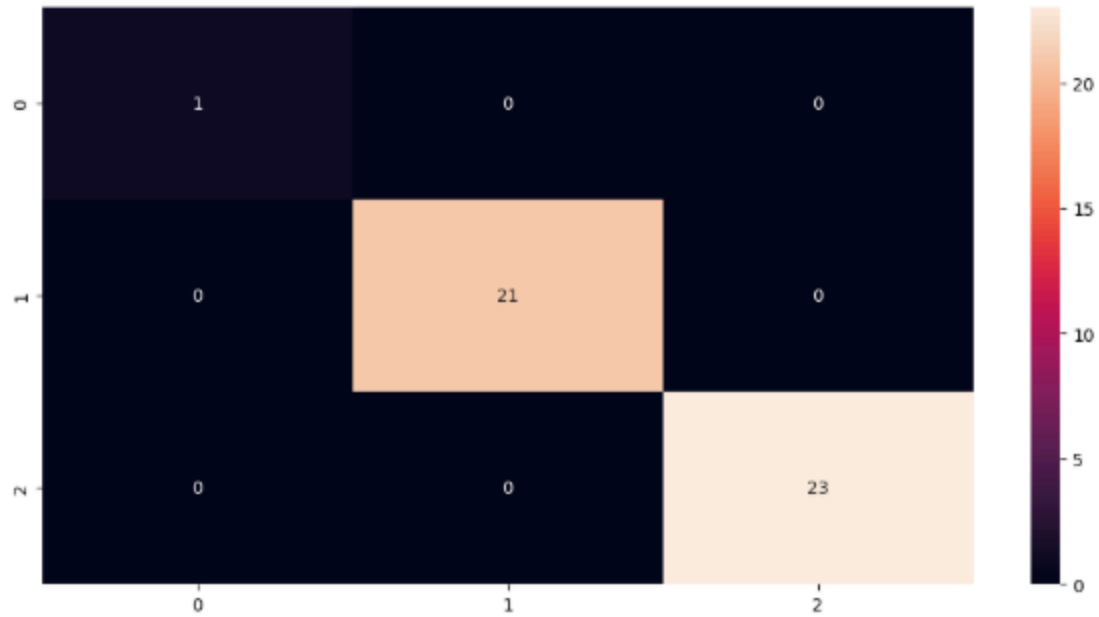

print("Interpretation: Tuning reduces overfitting and improves
generalization.")
```

Best Params: {'max_depth': 5, 'min_samples_split': 2}

Accuracy: 1.0

```
[[ 1  0  0]
 [ 0 21  0]
 [ 0  0 23]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	21
2	1.00	1.00	1.00	23
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45



```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report


df = pd.read_csv("/content/sales_data_sample.csv",encoding='latin1')

df = df.dropna()


le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == 'object':

        df[col] = le.fit_transform(df[col])


X = df.iloc[:, :-1]

y = df.iloc[:, -1]


X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,random_state=42)
```

```
model = RandomForestClassifier(n_estimators=100,random_state=42)

model.fit(X_train,y_train)


y_pred = model.predict(X_test)


acc = accuracy_score(y_test,y_pred)


print("Accuracy:",acc)

print(confusion_matrix(y_test,y_pred))

print(classification_report(y_test,y_pred))


plt.figure(figsize=(12,6))

sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,fmt='d')

plt.show()


importances = model.feature_importances_

features = X.columns


plt.figure(figsize=(10,5))

sns.barplot(x=importances,y=features)

plt.show()
```

```

if acc>0.9:

    print("Interpretation: High accuracy with reduced overfitting due to
ensemble learning.")

else:

    print("Interpretation: Performance can be improved with tuning.")

```

```

*** Accuracy: 0.9777777777777777

```

```

[[ 0  1  0]
 [ 0 21  0]
 [ 0  0 23]]

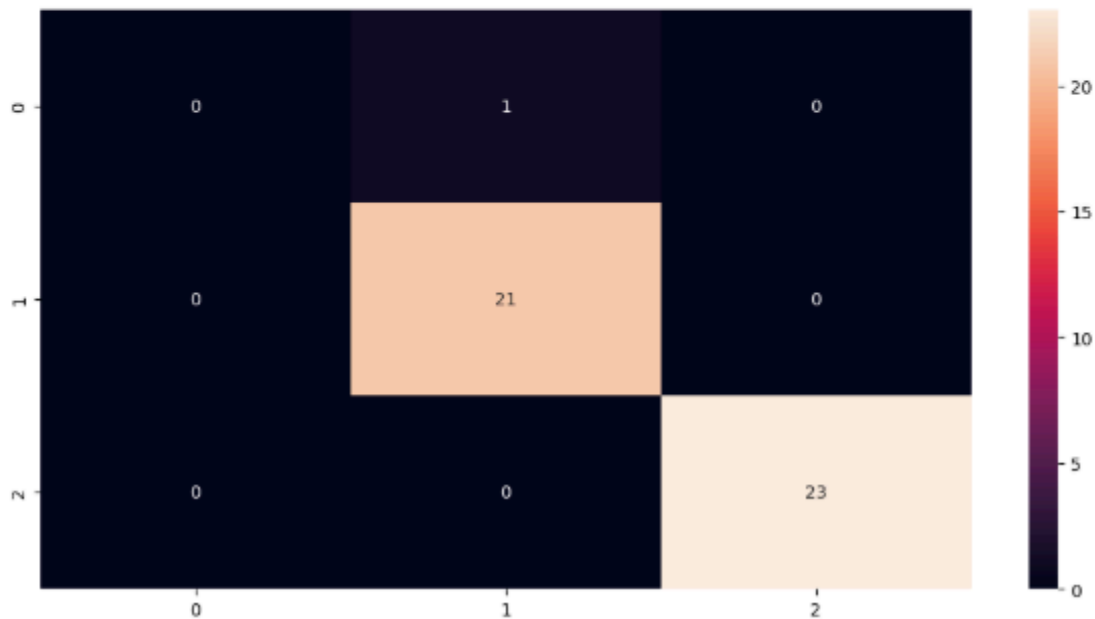
```

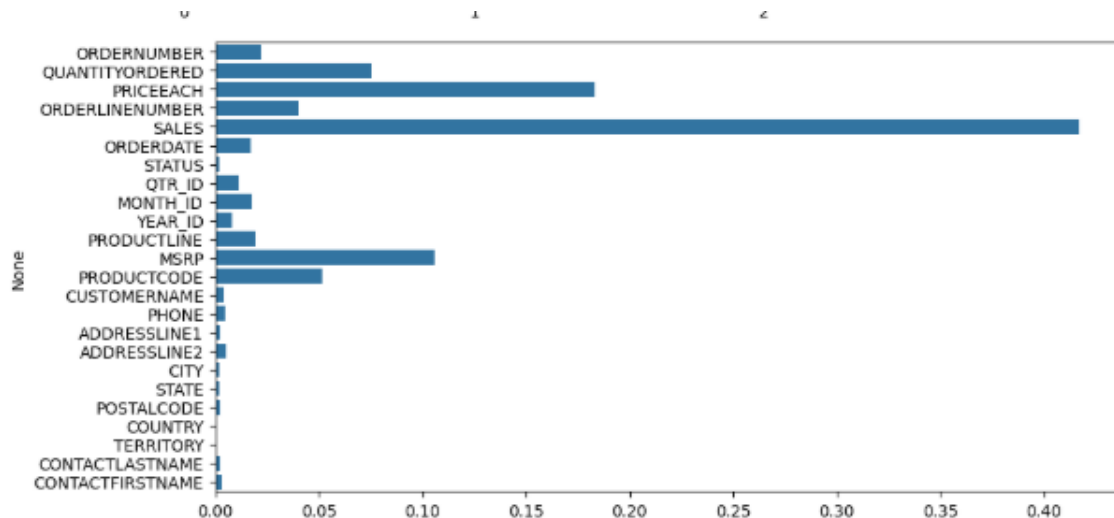
	precision	recall	f1-score	support
0	0.00	0.00	0.00	1
1	0.95	1.00	0.98	21
2	1.00	1.00	1.00	23
accuracy			0.98	45
macro avg	0.65	0.67	0.66	45
weighted avg	0.96	0.98	0.97	45

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```





Interpretation: High accuracy with reduced overfitting due to ensemble learning.

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pd.read_csv("/content/sales_data_sample.csv",encoding='latin1')

df = df.dropna()

le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == 'object':
```

```
df[col] = le.fit_transform(df[col])

X = df.iloc[:, :-1]

y = df.iloc[:, -1]

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,random_state=42)

params = {'n_estimators':[50,100,200], 'max_depth':[5,10,20,None]}

grid = GridSearchCV(RandomForestClassifier(random_state=42),params,cv=5)

grid.fit(X_train,y_train)

model = grid.best_estimator_

y_pred = model.predict(X_test)

acc = accuracy_score(y_test,y_pred)

print("Best Params:",grid.best_params_)

print("Accuracy:",acc)

print(confusion_matrix(y_test,y_pred))

print(classification_report(y_test,y_pred))
```



```
plt.figure(figsize=(12,6))
```

```
sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,fmt='d')
```

```
plt.show()
```

```
importances = model.feature_importances_
```

```
features = X.columns
```

```
plt.figure(figsize=(10,5))
```

```
sns.barplot(x=importances,y=features)
```

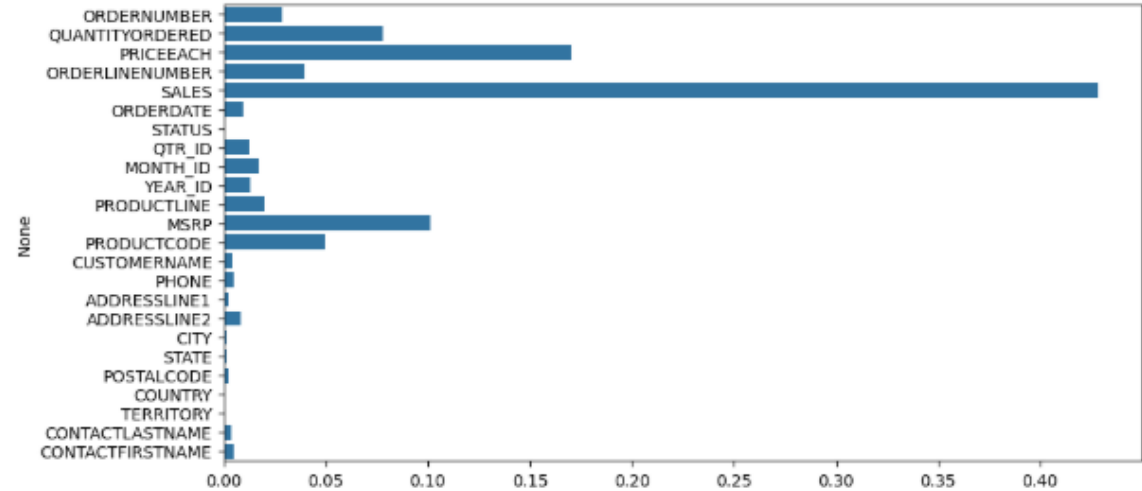
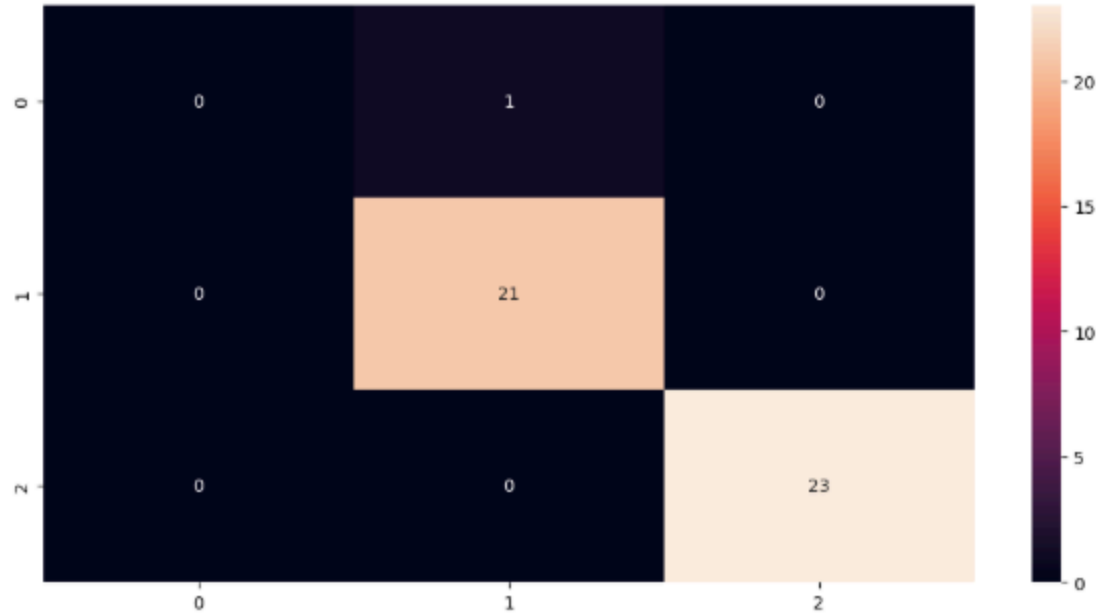
```
plt.show()
```

```
print("Interpretation: Tuned Random Forest gives best accuracy and  
stability with reduced overfitting.")
```

Best Params: {'max_depth': 5, 'n_estimators': 50}
Accuracy: 0.9777777777777777
[[0 1 0]
[0 21 0]
[0 0 23]]

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1
1	0.95	1.00	0.98	21
2	1.00	1.00	1.00	23
accuracy			0.98	45
macro avg	0.65	0.67	0.66	45
weighted avg	0.96	0.98	0.97	45

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))



Final Conclusion

- Decision Tree provides simple and interpretable results
- Random Forest improves accuracy using ensemble learning
- Sales classification helps businesses understand revenue segments
- Tree-based models are effective for business prediction tasks